Custom CUDA Kernels for GLM-ASR and RAG System Investigation

Anonymous authors
Paper under double-blind review

1 Introduction

This report covers INFR11269 coursework on machine learning systems: implementing the GPU kernels behind GLM-ASR-Nano speech recognition, then investigating retrieval-augmented generation (RAG) built on that stack. The ASR model routes every heavy operation through four CUDA tasks; PyTorch holds tensors and orchestrates the graph. Task 1 implements rotary position embedding (RoPE)—an element-wise kernel whose difficulty is index arithmetic over [batch, seq_len, heads, head_dim] layouts, not floating-point novelty.

The baseline is the supplied skeleton with empty blanks and PyTorch fallbacks (./run.sh –cuda none). We filled all four kernel groups, verified each with make test1 through make test4, and checked the full model with ./run.sh (all CUDA groups). The RAG extension is still outstanding; this report documents the ASR kernel work first.

2 System and Technical Approach

2.1 Supplied baseline

The homework provides csrc/task*.cu with numbered blanks, launch_* grid configuration, CPU references in tests/, and glm_asr_cuda/ops.py which selects CUDA or PyTorch per kernel group.

2.2 Task 1: RoPE kernel design

RoPE rotates pairs $(v[i], v[i+h])$ before attention, with $h = \text{rotary_dim}/2$: $\angle = \text{pos} \cdot \text{inv_freq}[i]$, $v[i]' = a\cos\angle - b\sin\angle$, $v[i+h]' = b\cos\angle + a\sin\angle$. One launch covers both q and k: threads $0, \ldots, \text{q_pairs} - 1$ touch q, then k. Each thread maps a global pair index to row, pair index i, and token position pos = (row/num_heads) mod seq_len. Elements beyond rotary_dim are untouched (audio encoder uses rotary_dim= 32 of head_dim= 64).

Implementation choices: ROPE_THREADS= 256 blocks sized by ceil(total_pairs / 256); in-place updates; cosf/sinf computed per pair (no lookup table). Blanks 1.1–1.4 implement indexing and the rotation; launch_rope is unchanged.

2.3 Task 2: warp/block reductions, RMSNorm, softmax

Part A (task2_reduce.cuh) implements warp_reduce_sum/max with ___shfl_xor_sync and block_reduce_sum/max via a partial[] staging buffer and two barriers. Part B builds rmsnorm_kernel (row-wise RMS scale with learned weights) and softmax_kernel using the online MaxSum merge from the guide so each row is read once. Task 4 reuses the warp helpers for tile-wise probability sums.

2.4 Task 3: shared-memory tiled GEMM

Following the lecture tiled GEMM, each 16×16 thread block computes a tile of $y = \text{GELU}(xW^\top + b)$ with TILE=16, loading x and transposed W into ___shared___ buffers

each K-step. Padding ws with [TILE][TILE+1] avoids bank conflicts; out-of-bounds loads use zero so every thread hits both ___syncthreads() calls.

2.5 Task 4: tiled attention with online softmax

Each block covers ATTN_WARPS=4 queries of one head; each warp streams keys/values in tiles of 32 through dynamic shared memory (k_tile, v_tile, cached scaled queries). Running statistics (m, ℓ, acc) implement FlashAttention-style online softmax with corr= $\exp(m - m_{\text{new}})$ when a tile raises the max. Causal masking sets scores to $-\infty$ past last_key; GQA maps kv_h = h / (num_heads/num_kv_heads). Blank 4.5 must broadcast each lane's probability with ___shfl_sync so every lane accumulates $\sum_r p_r v_r$ across all keys in the tile (not only its own lane).

3 Experimental Methodology

Hardware: NVIDIA RTX A6000 (sm_86, 84 SMs; test harness reports ∼768 GB/s peak DRAM bandwidth).

Software: mls-asr conda env (Python 3.11, PyTorch 2.10+cu128, transformers 5.0); CUDA via /usr/local/cuda; repository edin-mls-26-autumn/hw1-asr.

Quality metrics: per-case max_err against CPU references (make test1–4); transcription string match on test_audio.wav (./run.sh with all CUDA kernel groups).

Cost metrics: per-kernel latency or GFLOP/s from tests; end-to-end ms/token and –profile operation shares on the full model.

Controls: same GPU for all sweeps; rebuild with make clean when changing NVCCFLAGS; default -O3 -std=c++17 -lineinfo except for the –use_fast_math experiment.

4 Results and Analysis

4.1 Task 1 correctness and end-to-end check

All four make test1 cases pass with max_err $\leq 2 \times 10^{-7}$ (default flags). ./run.sh –cuda task1 yields 100% transcription accuracy (Concord returned to its place amidst the tents.) with rope=cuda and remaining groups on PyTorch.

4.2 Report questions (Task 1 guide)

Q1 (encoder bandwidth). The harness counts bytes moved as $2 \cdot (\text{q_rows} + \text{k_rows}) \cdot \text{rotary_dim} \cdot 4$ (read + write over the rotated prefix). For the audio-encoder case (B=1, seq=1500, H_q=H_k=20, rotary_dim=32): q_rows=k_rows=30,000, so $2 \cdot 60{,}000 \cdot 32 \cdot 4 = 15.36 \times 10^6$ bytes (∼14.6 MiB) per call. At ∼0.026 ms this is ∼601 GB/s, about 78% of the quoted 768 GB/s peak—consistent with a memory-bound kernel that also executes trigonometry and indexing.

Q2 (precomputed cos / sin table). A table of shape [seq_len, h] for cos and sin would add $2 \cdot \text{seq_len} \cdot h \cdot 4$ bytes read per pair (two floats) on top of existing q/k traffic, while removing FLOPs for cosf/sinf. For the encoder case (seq_len=1500, h=16) that is 192 KiB of table data but +8 bytes/pair on the critical path. At ∼78% of peak already, extra reads likely slow the kernel unless angles were reused many times without touching q/k (they are not). Tables help when trigonometry dominates and memory headroom exists; here arithmetic is cheap relative to streaming q and k.

Q3 (one launch for q and k). The model invokes RoPE 396 times per transcription. Fusing q and k halves kernel-launch count versus separate launches, avoids an extra global synchronisation point, and keeps one grid configuration hot in the driver. Cost is slightly more complex indexing (blank 1.2), which is negligible compared with launch latency (∼microseconds per launch) accumulated hundreds of times.

Table 1: RoPE encoder-case bandwidth vs. ROPE_THREADS (RTX A6000).

ROPE_THREADS	Time (ms)	Effective bandwidth
32	0.026	581 GB/s, 0.76 of peak
64	0.026	598 GB/s, 0.78 of peak
128	0.025	606 GB/s, 0.79 of peak
256	0.026	601 GB/s, 0.78 of peak
512	0.026	599 GB/s, 0.78 of peak
1024	0.026	591 GB/s, 0.77 of peak

4.3 Try this experiments (Task 1)

Table 1 sweeps ROPE_THREADS on the encoder case (default NVCCFLAGS). Throughput is flat near 76–79% of peak for block sizes 64–512; 1024 drops slightly (fewer blocks, less latency hiding). 32 is slowest (about 76% of peak): many small blocks increase launch and scheduling overhead relative to useful memory work.

Tiny cases (∼0.002 ms). Micro-benchmarks move only hundreds of floats; time is dominated by fixed launch latency and synchronisation, not by bytes transferred, so reported GB/s is near 0% of peak even when correct.

–use_fast_math. Rebuilding with NVCCFLAGS including –use_fast_math leaves tiny and decoder cases passing but fails the audio-encoder case (max_err≈1.9×10^{-4} vs. tolerance 10^{-5}). Encoder tokens reach pos ↑ 1499 with inv_freq[0]=1, so ∠ is large; hardware-style cosf/sinf approximations drift from the CPU reference. Decoder angles stay smaller (seq=64), so fast math still passes. I restored default flags after the experiment (make clean then make).

4.4 Tasks 2–4: tests and full ASR run

make test2 through make test4 all report ALL PASS on the RTX A6000 (encoder attention ∼8.6 ms, ∼1338 GFLOP/s in the harness). ./run.sh with every group on CUDA transcribes correctly (∼1961 ms per utterance vs. ∼475 ms with only Task 1 on CUDA—GEMM dominates). Profile: linear ∼82.5% of GPU time, attention ∼15.2%, remaining norms/activations under 2% each.

Task 2 (guide). Reductions combine per-thread partials without atomics; RMSNorm needs one block-wide sum of squares; online softmax avoids three passes over each row. Warp shuffles beat shared-memory trees inside a warp because they skip memory round-trips.

Task 3 (guide). Encoder fc1 alone is $1500 \times 5120 \times 1280$ MACs; tiled GEMM cuts global reads by reusing each loaded element across TILE threads. Our kernel reaches hundreds of GFLOP/s on large cases but remains below cuBLAS because we do not use tensor cores or CUTLASS.

Task 4 (guide). A dense encoder score tensor would be $20 \times 1500 \times 1500 \times 4 \approx 172$ MiB per layer; one block uses ∼48 KiB shared memory for a 32-key tile instead. All warps in a block must execute the same tile iterations because of ___syncthreads() between loads; per-warp early exit would deadlock. When a tile does not raise m, corr = 1; m_{new} stays $-\infty$ only before any valid score is seen—then corr is guarded to avoid NaNs. With four queries per block, each key is loaded once per four queries; larger ATTN_WARPS amortises loads but raises shared memory and register pressure.

5 Discussion and Conclusion

All four ASR kernels pass unit tests and the reference transcription on CUDA. RoPE is memory-bound; GEMM sets end-to-end latency; attention is the next largest cost but still streaming-friendly compared with materialising scores. The main implementation pitfall in Task 4 was treating each lane's p as local to its key when updating acc—cross-lane shuffle is

required. Fast math remains unsafe for RoPE at large angles. Next: RAG quality/latency experiments and optional comparison of CUDA ASR vs. PyTorch inside the RAG pipeline.

References

A ASR reproduction commands

```
conda activate mls-asr
export PATH=/usr/local/cuda/bin:$PATH
cd edin-mls-26-autumn/hw1-asr
make test          # or make test1 .. test4
./run.sh           # all CUDA kernels
./run.sh --profile
```

B RAG placeholder

RAG experiments and evaluation to be added when complete.

AI declaration

I used AI coding assistants (including Cursor/ELM) to help run GPU experiments, derive bandwidth accounting, and draft LaTeX text for this report. I reviewed all kernel code, re-ran make test and ./run.sh on my machine, and edited the final wording. I remain responsible for the submitted work.

Group information

Group ID: XXX

Information of group member A:

- Name: jacksu
- UUN: s2510156

Information of group member B:

- Name: TBD (partner)
- UUN: TBD (partner)